MASARAK / MILESTONE 2 / STAGE 2

# Jetpack Components and Libraries

Planned libraries, their use in MASARAK, and the reasons for choosing them.

## Component plan

01 - 06 / 12 libraries

| LIBRARY | USED FOR IN MASARAK | WHY WE CHOSE IT |
|---|---|---|
| **Jetpack Compose UI** | Building all MASARAK screens and reusable UI components. | It is the project's main declarative Android UI framework and allows the approved prototype to be implemented directly as composable screens. |
| **Material 3** | Buttons, text fields, cards, navigation bars, tabs, dialogs, bottom sheets, snackbars, chips, and the date picker. | It matches the approved MASARAK design system and provides consistent, accessible Android components with less custom UI code. |
| **Activity Compose** | Hosting the Compose application from the single MainActivity using setContent(). | It provides the standard bridge between an Android Activity and Jetpack Compose and fits the planned single-activity structure. |
| **Navigation Compose** | Navigating between Student, Field Supervisor, and Academic Supervisor screens and passing IDs such as taskId, traineeId, and studentId. | It provides structured navigation between composable destinations and supports MASARAK's role-specific flows. |
| **Lifecycle ViewModel Compose** | Obtaining and using ViewModels from composable screens. | It connects Compose screens to ViewModels in a lifecycle-aware way and keeps screen logic outside the UI code. |
| **Lifecycle Runtime Compose** | Collecting StateFlow values in Compose using lifecycle-aware state observation. | It prevents unnecessary UI collection when a screen is not active and supports reliable state-driven interfaces. |

# Jetpack Components and Libraries

Planned libraries, their use in MASARAK, and the reasons for choosing them.

## Component plan continued

07 - 12 / 12 libraries

| LIBRARY | USED FOR IN MASARAK | WHY WE CHOSE IT |
|---|---|---|
| **Kotlin Coroutines + Flow / StateFlow** | Asynchronous loading, submission, updates, chat operations, and exposing UI state from ViewModels. | It keeps the UI responsive and works naturally with ViewModel and Compose for reactive updates to tasks, attendance, evaluations, and messages. |
| **Firebase Authentication** | Sign up, login, logout, and authenticated access for the three MASARAK roles. | It provides managed authentication without requiring the team to build and secure a custom authentication server. |
| **Cloud Firestore** | Storing shared app data such as users, role links, tasks, attendance records, weekly evaluations, notifications, and chat messages. | It provides a cloud database that can synchronize shared training data between students and supervisors with less backend setup. |
| **Firebase Storage** | Uploading and storing task evidence, weekly attendance reports, and images/files shared in chat. | It handles user-uploaded files separately from structured Firestore data and avoids the need for a custom file server. |
| **Coil for Compose** | Loading and displaying remote images such as profile photos, task evidence, and chat images. | It integrates directly with Jetpack Compose and efficiently handles image loading and caching from remote storage. |
| **JUnit + Compose UI Test** | Testing core logic and important Compose user flows in later milestones. | The course requires testing and polish in a later milestone, so these tools provide unit testing for logic and UI testing for key Compose interactions. |